

NAME : Rowthu Bhaskar Abhiram

ROLL NO : A24126552114

BRANCH : CSM

SECTION : B

COURSE NAME : FULL STACK WEB DEVELOPMENT

CODE : 23CM4121

TITLE OF THE EXPERIMENT : To create a JavaScript-based To-Do List web application using DOM manipulation, dynamic HTML element creation, and event handling.

DESCRIPTION:

This code creates a To-Do List web application using HTML, CSS, and JavaScript. The HTML structure provides a text input field, an Add Task button, and an empty task list where tasks are displayed. CSS is used to style the application by creating a centered container, buttons, input fields, task items, and completed-task effects. JavaScript makes the webpage dynamic by selecting DOM elements using `getElementById()`, reading the task entered by the user, and creating new HTML elements using `document.createElement()`. When the Add Task button is clicked or the Enter key is pressed, a new task is created and displayed in the list. Each task has Complete and Delete buttons. The Complete button uses `classList.toggle()` to mark a task as completed with a strikethrough effect, while the Delete button removes the task from the list using `remove()`. The `updateEmptyState()` function displays or hides the “No tasks available” message depending on whether the task list contains any tasks. Thus, the program demonstrates DOM manipulation, dynamic element creation, event handling, CSS styling, and JavaScript functions in an interactive To-Do List application.

PROGRAM :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>To-Do List Application</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f6f8;
      padding: 30px;
      display: flex;
      justify-content: center;
    }

    .container {
      background-color: #ffffff;
      width: 480px;
      padding: 25px;
      border-radius: 8px;
      box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
    }

    h1 {
      text-align: center;
      color: #333333;
      margin-top: 0;
    }

    .input-group {
      display: flex;
      gap: 10px;
      margin-bottom: 20px;
    }

    #taskInput {
      flex: 1;
      padding: 10px 12px;
      font-size: 14px;
      border: 1px solid #ccc;
      border-radius: 4px;
      outline: none;
    }

    #taskInput:focus {
      border-color: #007bff;
    }

    #addBtn {
      padding: 10px 16px;
      background-color: #007bff;
      color: white;
      border: none;
      border-radius: 4px;
      cursor: pointer;
      font-weight: bold;
    }

    #addBtn:hover {
      background-color: #0056b3;
    }

    #taskList {
      list-style: none;
      padding: 0;
    }
```

```

        margin: 0;
    }

    .task-item {
        display: flex;
        align-items: center;
        justify-content: space-between;
        background-color: #fafafa;
        padding: 10px 12px;
        margin-bottom: 8px;
        border: 1px solid #e0e0e0;
        border-radius: 4px;
    }

    .task-text {
        flex: 1;
        font-size: 15px;
        color: #333;
        word-break: break-word;
        margin-right: 10px;
    }

    /* Completed task style */
    .completed {
        text-decoration: line-through;
        color: #888888;
    }

    .btn-group {
        display: flex;
        gap: 6px;
    }

    .complete-btn {
        background-color: #28a745;
        color: white;
        border: none;
        padding: 6px 12px;
        border-radius: 4px;
        cursor: pointer;
    }

    .complete-btn:hover {
        background-color: #218838;
    }

    .delete-btn {
        background-color: #dc3545;
        color: white;
        border: none;
        padding: 6px 12px;
        border-radius: 4px;
        cursor: pointer;
    }

    .delete-btn:hover {
        background-color: #c82333;
    }

    #emptyMessage {
        text-align: center;
        color: #777777;
        font-style: italic;
        margin: 15px 0;
    }
}
</style>
</head>
<body>

<div class="container">
    <h1>My To-Do List</h1>

    <div class="input-group">
        <input type="text" id="taskInput" placeholder="Enter a task...">
        <button id="addBtn">Add Task</button>
    </div>

    <p id="emptyMessage">No tasks available.</p>
    <ul id="taskList"></ul>
</div>

<script>
    // 1. DOM Element Selection
    const taskInput = document.getElementById("taskInput");
    const addBtn = document.getElementById("addBtn");
    const taskList = document.getElementById("taskList");
    const emptyMessage = document.getElementById("emptyMessage");

    // Helper function to toggle empty state message
    function updateEmptyState() {
        if (taskList.children.length === 0) {
            emptyMessage.style.display = "block";
        } else {
            emptyMessage.style.display = "none";
        }
    }
}

```

```

// Function to create and append a new task item
function addTask() {
    const taskText = taskInput.value.trim();

    if (taskText === "") {
        alert("Please enter a task before adding.");
        return;
    }

    // 2. Dynamic Element Creation
    const li = document.createElement("li");
    li.className = "task-item";

    const textSpan = document.createElement("span");
    textSpan.className = "task-text";
    textSpan.textContent = taskText;

    const btnContainer = document.createElement("div");
    btnContainer.className = "btn-group";

    const completeBtn = document.createElement("button");
    completeBtn.className = "complete-btn";
    completeBtn.textContent = "Complete";

    const deleteBtn = document.createElement("button");
    deleteBtn.className = "delete-btn";
    deleteBtn.textContent = "Delete";

    // 3. Event Listeners for Dynamic Elements
    // Toggle task completion
    completeBtn.addEventListener("click", function () {
        textSpan.classList.toggle("completed");
    });

    // Delete task item
    deleteBtn.addEventListener("click", function () {
        li.remove();
        updateEmptyState();
    });

    // 4. Construct the DOM tree
    btnContainer.appendChild(completeBtn);
    btnContainer.appendChild(deleteBtn);

    li.appendChild(textSpan);
    li.appendChild(btnContainer);

    taskList.appendChild(li);

    // Clear input and update empty list message
    taskInput.value = "";
    taskInput.focus();
    updateEmptyState();
}

// 5. Event Listeners
addBtn.addEventListener("click", addTask);

// Allow adding task by pressing the 'Enter' key
taskInput.addEventListener("keydown", function (event) {
    if (event.key === "Enter") {
        addTask();
    }
});
</script>
</body>
</html>

```

OUTPUT :

